Introduction

Operating system: Windows 11

Programming language: Python 3.13

Python libraries: pycryptodome and os

IDE: Visual Studio Code

How to compile, run, and use my code:

To execute the program from start to finish, pycryptodome needs to be installed and then you can run the main.py file in the src folder. The main function executes every function for tasks 1-4 in order. So, you can run python src/main.py in the project root folder to see all task outputs.

To run each function independently, you can use the commands stated below for each task.

Task 1

From the project root, the following command calls the key generation function:
python -c "from src.main import key_generation; key_generation('data/skprf.txt', 'data/skaes.txt', 256)"

Where 256 is the lambda value, 'data/skprf.txt' is the path of the secret key file for PRF, and 'data/skaes.txt' is the path of the secret key file for AES. Security parameter lambda=256 is the default value.

```
-------TASK 1. KEY GENERATION FUNCTION--------------
PRF Secret key (sk1): 26ce61235416c2c8de2a850d23490a7fbf734489b6a0088df5ccc7214f80a087
AES Secret key (sk2): 9f6759b026acf5937854754082722e121b759774b8f5527e0fd7626707aba4bd
```

Task 2

From the project root, the following command calls the encryption function:
python -c "from src.main import encryption; encryption('data/data_files', 'data/ciphertextfiles', 'data/skprf.txt', 'data/skaes.txt', 'data/index.txt')"

Where 'data/data_files' is the path for the folder including all the plaintext files, 'data/ciphertextfiles' is the path for the folder including all the ciphertext files,

'data/skprf.txt' is the path of the secret key file for PRF, 'data/skaes.txt' is the path of the secret key file for AES, and 'data/index.txt' is the path for the encrypted index file.

```
-------TASK 2. ENCRYPTION FUNCTION--------------
encrypted index:
Encrypted keyword: 4e0fbe62beaabff4faf1df43b109a533
Files: ['c1.txt', 'c4.txt', 'c6.txt']
Encrypted keyword: e627d47a10b8392b6439ff5d99c63746
Files: ['c1.txt', 'c4.txt', 'c5.txt']
Encrypted keyword: fcd27cee518e471e9d8665ad25acf5ab
Files: ['c1.txt', 'c2.txt', 'c3.txt', 'c5.txt']
Encrypted keyword: eb2cd09fdaf2f45a50e9d40a0ccf5565
Files: ['c2.txt']
```

Task 3

From the project root, the following command calls the token generation function:
python -c "from src.main import token_generation; token_generation('packers', 'data/skprf.txt', 'data/token.txt')"

Where ‘packers’ is a keyword, 'data/skprf.txt' is the path of the secret key file for PRF, and 'data/token.txt' is the path of the token file.

```
-------TASK 3. TOKEN GENERATION FUNCTION--------------
keyword:  packers
generated token:  fcd27cee518e471e9d8665ad25acf5ab
```

Task 4

From the project root, the following command calls the search function:
python -c "from src.main import search; search('data/token.txt', 'data/ciphertextfiles', 'data/skaes.txt', 'data/index.txt', 'data/result.txt')"

Where 'data/token.txt' is the path of the token file, 'data/ciphertextfiles' is the path for the folder including all the ciphertext files, 'data/skaes.txt' is the path of the secret key file for AES, and 'data/result.txt' is the path for the result file.

```
-------TASK 4. SEARCH FUNCTION--------------
files found with the keyword: c1.txt c2.txt c3.txt c5.txt
c1.txt bengals steelers packers
c2.txt packers patriots
c3.txt packers
c5.txt steelers packers
```

Implementation

I completed the project by randomly generating two secret keys of 256 bits and sk1 was used for the PRF and sk2 was used for AES encryption and decryption. Then I encrypted each plaintext using an IV and padding. After that, I encrypted each key word with sk1 for PRF and then stored the encrypted keywords with the ciphertext filenames. Finally, the search function the token helps find the matching files.